

2025-2026 春 《数据结构与算法》 第二次实验题解

助教-陈孙一硕

A 社团纳新 (easy)

首先考虑一个简单版，如果仍然是每个班级只能有一个人去，但是每个人去的班级数量没有限制，那应该怎么做？很显然应该是对于某一个班级，谁去的收益高就派谁去。

但是现在有了限制，那应该怎么办？我们做一个简单的假设，假设所有班级最开始都是 L 同学去的，那么陈同学应当要从 L 同学手中抢走一半的班级。怎么抢呢？当然是优先拿走那些陈同学去了之后，增加的收益更多的班级。这就是按照二者差值进行排序的原理了。

时间复杂度 $O(n \log n)$ 。

```
#include<iostream>
#include<algorithm>
using namespace std;
#define int long long
int n,ans=0;
struct node{
    int x,y;
};
bool cmp(node n1,node n2){
    return n1.x-n1.y>n2.x-n2.y;
}
node lst[100005];
void solve(){
    cin>>n;
    for (int i=1;i<=n;i++) cin>>lst[i].x>>lst[i].y;
    sort(lst+1,lst+n+1,cmp);
    for (int i=1;i<=n/2;i++) ans+=lst[i].x;
    for (int i=n/2+1;i<=n;i++) ans+=lst[i].y;
    cout<<ans<<endl;
```

```

}
signed main(){
    solve();
    return 0;
}

```

B 块排序

首先想清楚一件事情：1 这个数必须在第一个块里面。也就是说，在找第一个块的时候必须一直向后，直到把 1 包含进来再做后续的考虑。因为要求块的数量最多，也就是每个块一定要尽可能的小。我们需要尝试尽早结束第一个块，那么什么时候需要停止？那就是从开始的位置到当前恰好是 $1 \sim k$ 的一个排列。因此我们需要动态更新最大值。按照这个思路去找每一个块，遍历一遍即可。

时间复杂度 $O(n)$ 。

```

#include<iostream>
using namespace std;
#define int long long
int n,lst[1000005],ans=0;
signed main(){
    cin>>n;
    for (int i=1;i<=n;i++) cin>>lst[i];
    int mi=1,mx=-1;
    for (int i=1;i<=n;i++){
        mx=max(mx,lst[i]);
        if (mx==i) ans++,mi=i+1;
    }
    cout<<ans<<endl;
    return 0;
}

```

C 跳排序

从宏观的角度上看，如果 $a[i] = j$ ，那么这个数应当回到下标为 j 的位置，需要移动的距离就是 $|i - j| = |a[i] - i|$ 。但是对于某个 x ，每个数一次只能移动 x 个位置。那么如果这

个数字要归位, x 必须是 $|a[i] - i|$ 的因数。这意味着, 想要让所有数全部归位, 给定的 x 必须是所有 $a[i] - i, i = 1, 2, \dots, n$ 的因数, 即它们最大公约数的因数。

我们需要注意一种特殊情况, 就是如果整个序列已经有序了, 那对于每个 x 都应当输出 YES (因为题目说了可以不操作)。一个办法是把 gcd 的初始值设置成 0, 这样对于每次询问, $g \% tmp$ 的值都是 0, 都会输出 YES. 没想到这个办法的同学, 特判一下也不困难。

时间复杂度 $O(n \log n + k)$ 。

```
#include<iostream>
using namespace std;
#define int long long
int n,k,lst[100005],g=0,tmp;
int gcd(int x,int y){
    if (x<y) swap(x,y);
    if (x%y==0) return y;
    return gcd(y,x%y);
}
void solve(){
    cin>>tmp;
    if (g%tmp==0) cout<<"YES"<<endl;
    else cout<<"NO"<<endl;
}
signed main(){
    cin>>n>>k;
    bool flag=0;
    for (int i=1;i<=n;i++){
        cin>>lst[i];
        if (lst[i]-i==0) continue;
        if (!flag) g=abs(lst[i]-i),flag=1;
        else g=gcd(g,abs(lst[i]-i));
    }
    while (k--) solve();
    return 0;
}
```

D 先頭としか交 $\square$ できません！

先翻译一下题面：只能和排头交换！

如果 $a[1] \neq 1$ 就把这个数字换到它应该到达的位置去（使用 swap）。否则，找一个还没归位的数换到下标为 1 的位置。重复上述操作直到整个序列有序为止。这个做法的时间复杂度也是 $O(n)$ ，但是常数比较大。本来这个做法是可以通过全部测试的，但是在 LJQ 同学的建议下，补充了大体量的测试数据，通过卡常的手段，把这个做法卡掉了。（或许优化一下，也可能能通过，但不是出题人期望的做法）

正解应当是找交换环。离散数学应该已经学过划分的概念，所有交换环恰好构成的是整个序列的一个划分。如果某个交换环大小为 k ，且包含下标为 1 的位置，那么处理这个环的操作次数应当是 $k - 1$ 。如果某个环大小为 k ，但不包含下标为 1 的位置，那么需要先花费一次操作，找环中的一个数和 1 先交换位置，此时环的大小变成了 $k + 1$ ，就变成了第一种情况。所以总的操作次数是 $1 + (k + 1 - 1) = k + 1$ 。按照这个思路找环计数即可，时间复杂度 $O(n)$ 。

```
#include<iostream>
using namespace std;
#define int long long
int lst[1000005];
bool vis[1000005];
int ans=0;
signed main(){
    int n;cin>>n;
    for (int i=1;i<=n;i++) cin>>lst[i];
    for (int i=1;i<=n;i++){
        if (lst[i]==i||vis[i]) continue;
        int res=0,tmp=i;
        if (i==1) res--;else res++;
        while (!vis[tmp]) res++,vis[tmp]=1,tmp=lst[tmp];
        ans+=res;
    }
    cout<<ans<<endl;
    return 0;
}
```

E 四数交换排序

先找交换环. 如果是大小为 2 的环, 一次操作可以解决 2 个; 大小为 3 或 4 的环, 一次操作解决一个; 大小在 5 及以上的环, 每次操作可以让环的大小减少 3。按照这个思路, 先处理大小大于 5 的环, 直到其大小变为 2 或 3 或 4 为止。然后再处理大小为 2, 3, 4 的环即可。

时间复杂度 $O(n)$ 。

```
#include<iostream>
using namespace std;
#define int long long
int lst[1000005],cir[1000005],ans=0;
bool vis[1000005];
signed main(){
    int n;cin>>n;
    for (int i=1;i<=n;i++) cin>>lst[i];
    for (int i=1;i<=n;i++){
        if (lst[i]==i||vis[i]) continue;
        int res=0,tmp=i;
        while (!vis[tmp]) res++,vis[tmp]=1,tmp=lst[tmp];
        cir[res]++;
    }
    for (int i=5;i<=n;i++){
        ans+=i*cir[i]/3;
        if (i%3==2) cir[2]+=cir[i];
    }
    ans+=(cir[2]+1)/2+cir[3]+cir[4];
    cout<<ans<<endl;
    return 0;
}
```

F 最远的距离

不妨设 $i < j$ 。那么只有 $a_i < a_j$ 和 $a_i \geq a_j$ 两种情况。

复制两份数组。对第一份数组按照 $i^2 + a_i^2$ 进行排序, 第二份数组按照 $i^2 - a_i^2$ 进行排序。假设第一份数组中, 两端的数分别为 $x^2 + a_x^2, y^2 + a_y^2$, 且 $x < y$ 。

- 如果 $a_x < a_y$ ，那么说明这两个位置之间的 CQU 距离就是二者之差，按照 $i^2 + a_i^2$ 来排序是正确的；
- 否则，我们得出的 CQU 距离为 $|x^2 + a_x^2 - (y^2 + a_y^2)|$ ，而这两者之间的 CQU 距离实际上应该是 $|x^2 - a_x^2 - (y^2 - a_y^2)|$ ，显然后者是大于前者的，所以取 max 时，前者不会影响答案。

对于按照 $i^2 - a_i^2$ 排序的情况，分析同理。

因此，只要按照两种情况分别排序一遍，分别计算头尾差的绝对值，取最大即可。

时间复杂度 $O(n \log n)$ ，主要来源于排序的复杂度。

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'
const int N=2e6+101;
int a[N];
void solve(){
    int n; cin>>n;
    vector<int> f,b;
    for(int i=1;i<=n;i++) cin>>a[i];
    for(int i=1;i<=n;i++){
        f.push_back(a[i]*a[i]+i*i);
        b.push_back(a[i]*a[i]-i*i);
    }
    sort(f.begin(),f.end());
    sort(b.begin(),b.end());
    cout<<max(abs(f[0]-f[n-1]),abs(b[0]-b[n-1]))<<endl;
}
signed main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    int T=1; //cin>>T;
    while (T--) solve();
    return 0;
}
```

G 杂货店店主的烦恼

出这道题是为了向大家介绍一个方法：微扰法。

假设 $a_1, a_2, \dots, a_n$ 从下到上放置。类似冒泡排序的思想，我们分析何时 a_i 排在 a_{i+1} 的下面，是比将它们交换位置之后更优的。

由于每个物品只会影响放在它下面的物品的实际体积，因此，考虑 a_i 和 a_{i+1} 对体积压缩的贡献。

在不交换位置时， a_i 对体积压缩的贡献为：

$$\sum_{j=1}^{i-1} c_j w_i$$

a_{i+1} 对体积压缩的贡献为：

$$\sum_{j=1}^{i-1} c_j w_{i+1} + c_i w_{i+1}$$

如果将二者交换位置，同理可得到二者对压缩的总贡献为

$$\sum_{j=1}^{i-1} c_j w_{i+1} + \sum_{j=1}^{i-1} c_j w_i + c_{i+1} w_i$$

考虑交换前后的贡献差大于 0，得到

$$c_i w_{i+1} > c_{i+1} w_i$$

依照此规则重载小于运算符，对所有物品进行排序，随后用一个后缀记录，线性时间模拟计算答案即可。

时间复杂度 $O(n \log n)$ ，主要来源于排序。

```
#include <bits/stdc++.h>
#define int long long
using namespace std;
const int N=2e6+101;
struct node{
    int w;
    int v;
    int c;
};
bool cmp(node n1,node n2){
```

```

        return n2.c*n1.w<n1.c*n2.w;
    }
    node a[N];
    int suf[N];
    void solve(){
        int n; cin>>n;
        for (int i=1;i<=n;i++) cin>>a[i].w>>a[i].v>>a[i].c;
        sort(a+1,a+n+1,cmp);
        for (int i=n;i>=1;i--) suf[i]=suf[i+1]+a[i].w;
        int ans=0;
        for (int i=1;i<=n;i++){
            ans+=(a[i].v-a[i].c*suf[i+1]);
        }
        cout<<ans<<endl;
    }
    signed main() {
        ios::sync_with_stdio(false);
        cin.tie(nullptr);
        int T=1; //cin>>T;
        while (T--) solve();
        return 0;
    }
}

```

H 校队选拔

去年天梯赛重庆大学校选赛的真题，一道略为复杂的模拟，注意 20 位需要用 string 存储，剩下的慢慢调整去吧。

```

#include<iostream>
#include<cstdio>
#include<algorithm>
#include<vector>
#include<queue>
#include<map>
using namespace std;
#define LL long long

```



```

#define MAXM 50000
#define MAXN 100000
#define Pr pair<int,int>
#define X first
#define Y second
LL read(){
    LL x=0,F=1;char c=getchar();
    while(c<'0' || c>'9'){if(c=='-')F=-1;c=getchar();}
    while(c>='0'&&c<='9'){x=x*10+c-'0';c=getchar();}
    return x*F;
}
int n,m,tot;
map<string,int> D;
struct stu{
    string name,id;
    int score[11],tots;
}a[MAXN+5];
bool cmp(stu s1,stu s2){
    if(s1.tots==s2.tots)return s1.id>s2.id;
    return s1.tots>s2.tots;
}
int main(){
    n=read(),m=read();
    for(int i=1;i<=n;i++){
        string tname,tid;
        cin>>tname>>tid;
        int tsc=read(),tscore=read();
        int tt1,tt2;
        scanf("%02d:%02d",&tt1,&tt2);
        if(tt1<3){
            if(D.count(tid)==0){
                D[tid]=++tot;
                a[tot].name=tname;
                a[tot].id=tid;
            }
            int p=D[tid];

```

```

        a[p].tots-=a[p].score[tsc];
        a[p].score[tsc]=max(a[p].score[tsc],tscore);
        a[p].tots+=a[p].score[tsc];
    }
}
sort(a+1,a+n+1,cmp);
for(int i=1;i<=m;i++){
    cout<<a[i].name<<" "<<a[i].id<<" "<<a[i].tots<<"\n";
}
}

```

I 二元组排序

答案为 Yes 的充要条件是，对于 $\forall 2 \leq i \leq n$ ，都有 $(i-1, i)$ 的存在。充分性证明可以参考冒泡排序（或者说这就是个冒泡排序）。必要性：如果有 $(x-1, x)$ 不存在，将序列排序后，这两个位置颠倒，那么无论执行多少轮都无法完成排序。

```

//#include<bits/stdc++.h>
#include<iostream>
#include<iomanip>
#include<vector>
#include<string>
#include<cmath>
#include<algorithm>
#include<set>
#include<map>
#include<unordered_map>
#include<unordered_set>
#include<queue>
#include<stack>
using ld = long double;
using i64 = long long;
using namespace std;
void solve(){
    int n,m; cin>>n>>m;
    set<pair<int,int>> ss;

```

```

while (m--){
    int a,b; cin>>a>>b;
    ss.insert({a,b});
}
for (int i=2;i<=n;i++) {
    if (!ss.count({i-1,i})){
        cout<<"No\n";
        return;
    }
}
cout<<"Yes\n";
}
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    int T=1; cin>>T;
    while (T--) solve();
    return 0;
}

```

J 逆序对

签到题。本来想搞个 $n \leq 2 \times 10^5$ 的版本让大家用归并排序来做（或者树状数组也行）。但怕有人爆零，蒜鸟蒜鸟，都不容易，暴力做法就行。（这题应该不用放代码了吧）